perplexity

Overall assessment

Yes. The document has a strong conceptual core, but it currently mixes **principles**, **design rules**, **implementation/specification details**, **rationale**, and **open decisions**. That makes it hard to tell which statements are foundational and which are provisional. The best reduction is not just to merge numbered items: it is to separate those layers, then keep a short set of principles and move operational detail elsewhere. [1]

Redundancy and opportunities to merge

Several ideas recur across the organizers and the L-principles:

- **G0, G1, and L2** all center on types, propositions, and membership as a guarantee. G1 is the clearest core claim; G0 can explain its consequences for declaration and execution, while L2 can specify what "guarantee" means at different certificate grades.
- **G3 and L10** both say that demand determines what is computed. Keep one principle, with the detailed planner behavior—backward planning, live subgraphs, memo keys, and output decorations—in a specification or design note.
- **L5, L7, L13, and L14** all address equality, certification, and when one implementation may stand in for another. They could be organized under one higher-level principle—*substitution requires evidence of equivalence*—with separate rules for identity, verification, transport, and planner eligibility.
- **L2, L7, L10, and L14** repeat that certificates carry evidence and grades, travel with results, and constrain reuse. State this once as a certificate principle; put certificate formats, hashing, caching, and invalidation in the spec.
- **L9 and L11** share a useful distinction between pure evaluation and actions on the world. L9 can state the pure dataflow model; L11 can be a short corollary for actions and their verification.
- **L3 and L4** both describe construction from declarations and laws. They are related, but I would not merge them completely: one concerns certified types and constructors, the other holes, declarations, and elaboration.

These changes could plausibly reduce the **15 L-principles to about 8–10**, without losing substance. The three G-principles should remain only if they are genuinely a separate, higher-level framing; otherwise, they duplicate the L-principles and add another numbering system.

Tensions and apparent contradictions

Some are real design questions; others may be resolved by clarifying terms.

- **"A model's law is certified" versus tests as evidence:** The terms define a model as a value whose law is certified, but the certificate may be sampled or assumed. L2 partly repairs this by qualifying "guarantee," yet the word *model* can still sound stronger than the evidence warrants. Consider distinguishing "candidate model" from "certified model," or explicitly defining modelhood as relative to a certificate grade.
- **"One function, many equivalent implementations" versus non-transitive closeness:** L13 recognizes that tolerance-based closeness is not transitive and later describes a weighted graph rather than a true equivalence class. Calling the implementations a *quotient* or a *class* is therefore misleading in the approximate case. Reserve "equivalence class" for genuinely transitive equality, and call the approximate case a graph of admissible substitutions.
- **"One law / one certificate" versus compound laws:** L6 says a theory has one law and one certificate, while also allowing laws to combine and their grades to be composed. Clarify whether the certificate is a single record containing a structured set of component evidence, or whether composition literally produces one certificate.
- **"Statements commute" versus textual binding order:** L9 says statements commute, then says order inside a block determines what a name means. These can coexist if the claim is restricted to *independent definitions after name resolution*. State that restriction in the headline or first sentence.
- **"Every value is certified by default" versus assumed values and opt-out:** L14's default is certification, but it also permits named worlds and an opt-out that leaves an `assumed` certificate; the opt-out syntax is open. Define whether assumed status is a certificate, a missing certificate, or a separate state. Avoid saying "certified" without qualification when the grade is `assumed`.
- **"An action is verified by before-and-after observation" versus external irreversible or concurrent effects:** L11 says these effects are logged and audited, "never verified." That is too categorical unless *verified* means verified solely from observable state. Say what evidence can be established and what cannot—for example, that a log records an attempted action but does not prove its external effect.
- **The type guides declarations, but demand governs execution:** G0 and G3 are compatible in principle, but the division needs a crisp statement: types and laws constrain admissible results; demand selects which result, evidence, and resources are required.
- **"Only what leaves the program carries a certificate" versus certification as a default for every value:** L2/L10 and L14 can be reconciled by distinguishing internal values from demanded outputs and their dependency cones. State this once, explicitly.

Missing principles or explicit rules

The draft already lists many unresolved questions, so these are gaps in the *principles document's architecture* rather than claims that the design has no answer.

- **Trust and threat model:** L15 distinguishes the kernel, libraries, host runtime, and planner, but the document should state which components are trusted, what each certificate attests to, and what a compromised or nondeterministic host invalidates.
- **Soundness boundary:** Define what each grade supports. In particular, "sampled at a level" needs a precise interpretation of the level, distribution, covered inputs, and whether it is a confidence statement or simply a record of tests run.
- **Failure and partial results:** State what happens when a law fails, a certificate is unavailable, a cost cannot be composed, or a demanded tolerance cannot be met. "Refused," "error," "assumed," and "no substitution" appear, but there is no compact rule tying them together.
- **Effect and reproducibility boundary:** The document distinguishes pure evaluation from actions, but should state how nondeterminism, concurrency, external state, and repeated actions affect result identity and caching.
- **Terminology and normative status:** The draft uses "settled," "proposed," "candidate," and "open," sometimes within the same principle. Add a status legend and make the published principles contain only approved commitments—or visibly label tentative claims.

A better document structure

I'd split the current file into four documents:

1. **Principles:** Short, stable claims, each with a one-sentence statement and a brief rationale.
2. **Specification:** Syntax, typing rules, certificate formats, planner behavior, caching, action semantics, and error handling.
3. **Rationale and design notes:** Motivations, examples, alternatives considered, citations, and historical discussion.
4. **Open decisions:** Only unresolved questions, each with an owner or decision needed and links to the affected principle/spec section.

That would also solve the draft's most immediate readability issue: L10 and L13 contain so much machinery that their headline claims are difficult to locate. Keep the detail, but move it out of the principles layer.[1]

Possible reduced set

A compact target could be **eight principles**:

1. **The language expresses the invariances that define scientific concepts.**
2. **Types describe structures and laws; values are models, with evidence stated at an explicit grade.**
3. **Laws generate certified types, constructors, and membership checks.**
4. **Substitution requires a certificate of identity or equivalence, with its scope, grade, and tolerance preserved.**
5. **Programs describe pure dependencies; effects are explicit actions.**
6. **Demand determines which result, model, evidence, and resources are produced.**

7. **The planner chooses among admissible implementations according to the demander's declared trade-offs.**
8. **The core is small and explicit; higher-level facilities reduce mechanically to it, and programs are data.**

Then treat **cost-valued laws** as a central design extension if it is the project's defining contribution, rather than burying its still-open algebra inside a supposedly settled principle. Treat declarations/holes, action verification, parameters, foreign-language delegation, and specific caching rules as specification rules or corollaries—not all as equally foundational principles. [1]

⁂

1. principles.md